

## 박정환

Backend Developer · Full-Stack · AI Service

pjh5929@naver.com

010-7185-3377

총 경력 4년

## 1 PROJECT 01 · (주)가비아

## 전자결재 차세대 REST API 개발

2022.08 - 2024.10 · 2년 2개월

역할: 파트장 · 백엔드 설계·구현 총괄

기여도: 백엔드 단독 주도

Java 17

Spring Boot 3

MySQL / Aurora

MongoDB

Kafka

Docker

Kubernetes

JUnit5

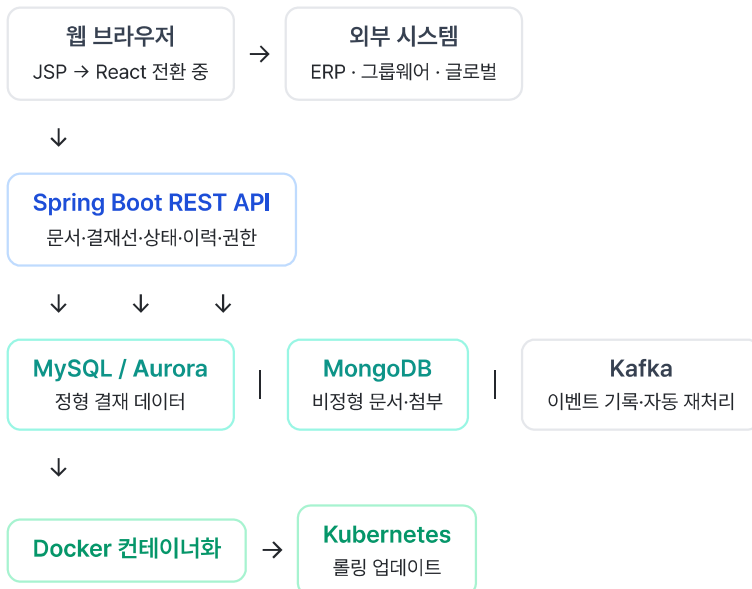
Mockito

ERP · 그룹웨어 연동

10년 이상 운영된 PHP 모놀리식 전자결재 시스템을 **Java / Spring Boot REST API 기반 차세대 구조**로 전환하는 프로젝트. 기능 확장성·성능·운영 안정성을 목표로 MSA 방향의 아키텍처 설계·구현을 파트장으로서 단독 주도했습니다.

## 시스템 아키텍처

## 전체 흐름



## 핵심 문제 해결

① 트랜잭션 범위 재설계 — API 응답 97% 개선

| PROBLEM                                                        | SOLUTION                                                       | RESULT                                                           |
|----------------------------------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------|
| 로그 분석 결과 트랜잭션 범위가 불필요하게 넓어 DB 락 과도 유지. 외부 API 호출까지 트랜잭션 내에 포함. | 데이터 정합성이 실제로 필요한 구간에만 트랜잭션 적용하도록 재설계. 외부 I/O 분리. 성능 테스트 직접 주도. | 평균 API 응답 3,000ms → 100ms (97% 개선). DB 락 경험 감소, 동시 요청 처리 안정성 향상. |

## ② Kafka 기반 자동 복구 구조 — 수동 복구 0건 달성

| PROBLEM                                                 | SOLUTION                                                              | RESULT                                     |
|---------------------------------------------------------|-----------------------------------------------------------------------|--------------------------------------------|
| MSA 전환 중 네트워크 단절 시 처리 실패 이벤트를 수동으로 재처리하는 작업이 월 수십 건 반복. | Kafka로 모든 처리 이벤트 기록, 네트워크 정상화 시 배치가 자동 재처리. 장애 시 데이터 유실 없는 역등성 보장 구조. | 수동 복구 월 수십 건 → 0건. 운영 개입 없이 데이터 정합성 자동 유지. |

## ③ 인증·인가 체계 중앙화 — 보안 리스크 감소

| PROBLEM                                                             | SOLUTION                                                        | RESULT                                     |
|---------------------------------------------------------------------|-----------------------------------------------------------------|--------------------------------------------|
| 역할·문서 상태·조직 관계에 따라 행위별 권한이 복잡하게 달라지는 도메인. 기존 코드는 권한 로직이 각 API마다 중복. | 인증은 공통 필터, 인가는 도메인 서비스 레벨에서 중앙화. 권한 로직 한 곳 관리, 신규 API 시 재사용 구조. | 보안 리스크 감소, 인가 로직 재사용으로 개발 효율 및 유지보수 비용 절감. |

## 성과 지표

|                                                      |                                                  |                                              |                                              |
|------------------------------------------------------|--------------------------------------------------|----------------------------------------------|----------------------------------------------|
| 3,000ms<br><b>100ms</b><br>트랜잭션 재설계<br>API 응답 97% 개선 | 수십 권/월<br><b>0건</b><br>Kafka 도입 후<br>수동 복구 완전 제거 | <b>멀티 DB</b><br>MySQL + MongoDB<br>목적별 분리 설계 | <b>JUnit5</b><br>단위·통합 테스트 도입<br>회귀 버그 방지 체계 |
|------------------------------------------------------|--------------------------------------------------|----------------------------------------------|----------------------------------------------|

## 2 PROJECT 02 · (주)가비아

# 전자결제 유지보수 / 고도화

2020.11 – 2024.10 · 4년

역할: 파트장 · 레거시 백엔드 운영 및 성능 개선

PHP

CodeIgniter

Laravel

MySQL / Aurora

Linux

DB Partitioning

Index Tuning

사내 전자결제 레거시 서비스 백엔드 개발·운영 담당. **PHP 모놀리식 구조를 안정적으로 운영**하면서 기능 고도화·대용량 데이터 성능 개선을 지속하고, 차세대 REST API 전환을 위한 구조적 기반을 마련했습니다.

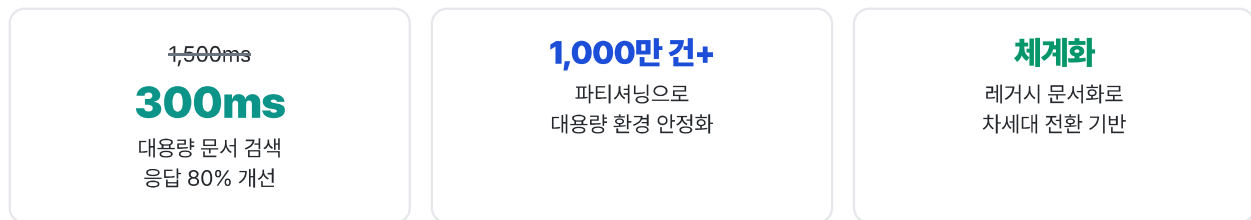
## 핵심 문제 해결 — 대용량 문서 검색 성능 개선

- **문제:** 전자결재 문서 1,000만 건+ 누적 → 문서 검색·목록 조회 성능 저하 지속 발생
- **분석:** 슬로우 쿼리 로그 분석 → Full Table Scan, 비효율적 서브쿼리, 인덱스 미활용 구간 특정
- **해결:** office\_no 기반 파티셔닝·샤딩 적용, 복합 인덱스 재설계, 불필요 조인·서브쿼리 제거 후 단계적 검증
- **결과:** 응답 시간 **1,500ms → 300ms** (80% 개선), 대용량 환경 조회 안정화

## 차세대 전환 기반 마련

- 10년 이상 축적된 전자결재 비즈니스 로직·예외 케이스·운영 노하우 **체계적 문서화**
- 신규 Java REST API 서버 도입을 위한 **연계 구조 설계** 및 전환 기반 마련
- 레거시·차세대 병행 운영 기간 동안 **데이터 정합성 유지** 구조 설계

## 성과 지표



## 3 PROJECT 03 · 개인 프로젝트

# 로스트아크 인벤 사건사고 검색기

2024.10 - 2026.03 · 1년 6개월

역할: 전체 설계·구현 단독

기여도: 100% (풀스택 + AI 서비스)

Java 17

Spring Boot 3.2

CompletableFuture

Jsoup

JUnit5

WireMock

React

TypeScript

Vite

Python

FastAPI

PaddleOCR

OpenCV

게임(로스트아크) 파티 구성 시 악성 유저를 자동으로 걸러내는 풀스택 웹 서비스. **React 프론트엔드 · Spring Boot 병렬 처리 백엔드 · Python AI OCR** 마이크로서비스 — 3개 서비스 레이어를 처음부터 단독으로 설계·구현했습니다.

## 전체 서비스 아키텍처

전체 파이프라인

프론트엔드 (REACT + TYPESCRIPT)

닉네임 검색 UI

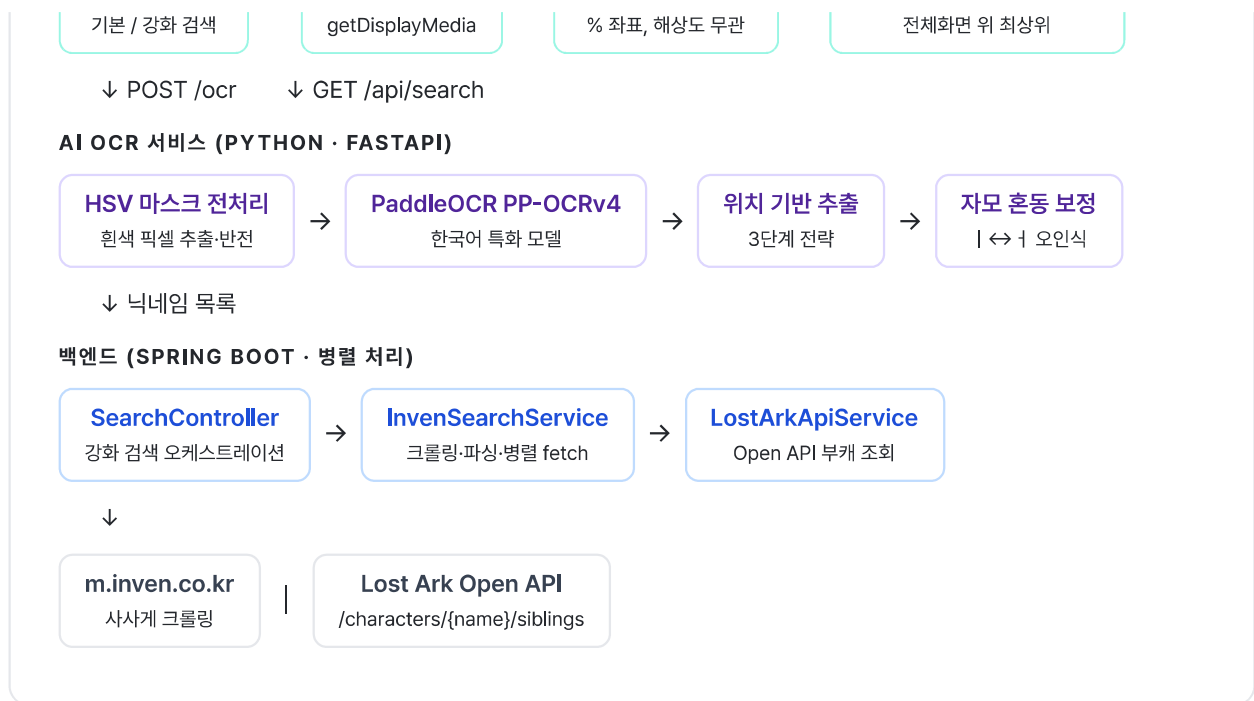
화면 공유 캡처

→

Canvas ROI 추출

→

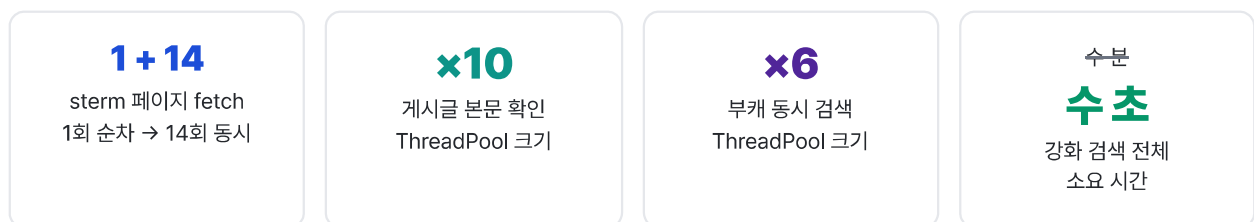
Document PiP 오버레이



## Spring Boot — 병렬 처리 최적화 상세

캐릭터 6개 × 15페이지 × 게시글 본문 확인 = 수백 건의 외부 HTTP 요청이 필요한 구조. sterm 파라미터가 +10,000 단위로 규칙적으로 증가한다는 것을 파악하여 나머지 페이지 URL을 선계산 후 동시 요청하는 구조로 전환했습니다.

```
// sterm 기준값 추출 → 나머지 14페이지 URL 선계산 후 동시 요청
long stermBase = Long.parseLong(firstSterm);
List<CompletableFuture<Document>> pageFutures = IntStream.range(1, maxIterations)
    .mapToObj(i -> {
        long sterm = stermBase + (long)(i - 1) * 10_000L;
        return CompletableFuture.supplyAsync(
            () -> fetchDocument(url + "&sterm=" + sterm), PAGE_EXECUTOR);
    }).collect(Collectors.toList());
// 게시글 본문 확인 - POST_EXECUTOR(×10), 부캐 검색 - CHAR_EXECUTOR(×6)
```



## AI OCR 마이크로서비스 상세

| 엔진                  | 결과   | 사유                                               |
|---------------------|------|--------------------------------------------------|
| Tesseract.js (브라우저) | 탈락   | 한국 게임 폰트 인식을 부족                                  |
| EasyOCR (Python)    | 탈락   | 범용 모델, 한글 정확도 한계                                 |
| PaddleOCR 2.7.x     | ✓ 채택 | 한국어 특화 PP-OCRv4 · Windows oneDNN 충돌로 2.7.x 버전 고정 |

위치 기반 닉네임 추출 — 3단계 전략

- **전략 1 (Lv.XX 오른쪽 탐색):** Lv.XX 텍스트와 같은 Y좌표, 오른쪽에 있는 유효 텍스트 추출 (우선 시도)
- **전략 2 (서버명 아래 탐색):** 서버명(실리안·루페온 등 8개 제외 목록)과 같은 X열, 바로 아래 텍스트 (전략 1 실패 시)
- **전략 3 (행 클러스터 탐색):** Y좌표 기준으로 행을 분류 후 유효 텍스트 탐색 (최후 풀백)

## 크롤링 신뢰도 확보

| PROBLEM                                                | SOLUTION                                                  | RESULT                                 |
|--------------------------------------------------------|-----------------------------------------------------------|----------------------------------------|
| style=content 검색은 닉네임이 게시글 어디에나 있으면 포함 → 무고한 게시글 다수 혼입 | 개별 게시글 fetch 후 "대상자:" 색션에만 닉네임 존재 여부 정밀 검증. 3단계 오탐 방지 로직. | 오탐 최소화. 실사용자 제보로 발견한 5가지 분리자 패턴 전부 처리. |

## React 프론트엔드 — 최신 브라우저 API 활용

- **Document Picture-in-Picture API:** 일반 팝업은 게임 전체화면에 가려지는 문제 → Chrome 116+ Document PiP API로 항상 최상위 플로팅 창 구현
- **화면 공유 OCR 파이프라인:** getDisplayMedia API → Canvas ROI 추출(% 좌표, 해상도 무관) → FastAPI 전송
- **버그 해결:** canvas 조건부 렌더링 내 ref null 발생 → display:none/grid로 가시성만 제어하는 방식으로 해결

## 성과 요약

|                                                                                                                                                                          |                                                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>병렬 처리 — 수 분 → 수 초</b><br>CompletableFuture + 3개 ThreadPool 구조로 수백 건 HTTP 요청 동시 처리 |  <b>AI OCR 마이크로서비스 구축</b><br>도메인 특화 전처리 + 위치 기반 추출로 한국 게임 폰트 인식을 확보                  |
|  <b>React SPA 단독 구현</b><br>백엔드 개발자로서 TypeScript SPA + 최신 브라우저 API 활용                  |  <b>멀티 서비스 파이프라인 통합</b><br>Spring ↔ React ↔ FastAPI ↔ Lost Ark API ↔ Inven 크롤링 단독 통합 |